

Balanced Binary Tree

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */

class Solution {
    public int height(TreeNode root){
        if (root == null) return 0;

        int leftHeight = height(root.left);
        if (leftHeight == -1) return -1;

        int rightHeight = height(root.right);
        if (rightHeight == -1) return -1;

        if (Math.abs(leftHeight - rightHeight) > 1) return -1;

        return 1 + Math.max(leftHeight, rightHeight);
    }
    public boolean isBalanced(TreeNode root) {
        return height(root) != -1;
    }
}
```

Delete Node in a BST :

```
/**
```

```

* Definition for a binary tree node.
* public class TreeNode {
*     int val;
*     TreeNode left;
*     TreeNode right;
*     TreeNode() {}
*     TreeNode(int val) { this.val = val; }
*     TreeNode(int val, TreeNode left, TreeNode right) {
*         this.val = val;
*         this.left = left;
*         this.right = right;
*     }
* }
*/

class Solution {
    public TreeNode deleteNode(TreeNode root, int key) {
        if (root == null) return null;

        if (key < root.val) root.left = deleteNode(root.left, key);
        else if (key > root.val) root.right = deleteNode(root.right, key);
        else {
            if (root.left == null) return root.right;
            if (root.right == null) return root.left;
            TreeNode successor = root.right;
            while (successor.left != null) {
                successor = successor.left;
            }
            root.val = s/**
* Definition for a binary tree node.
* public class TreeNode {
*     int val;
*     TreeNode left;
*     TreeNode right;
*     TreeNode() {}
*     TreeNode(int val) { this.val = val; }
*     TreeNode(int val, TreeNode left, TreeNode right) {
*         this.val = val;
*         this.left = left;
*         this.right = right;
*     }
* }
*/

class Solution {
    public int kthSmallest(TreeNode root, int k) {
        ArrayList<Integer> list = new ArrayList<>();
        inorder(root, list);
        return list.get(k - 1);
    }
}

```

```

    }

    public void inorder(TreeNode root, ArrayList<Integer> list) {
        if (root == null) return;
        inorder(root.left, list);
        list.add(root.val);
        inorder(root.right, list);
    }
}
successor.val;
    root.right = deleteNode(root.right, successor.val);
}
return root;
}
}

```

Kth Smallest Element in a BST :

```

/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */

```

```

class Solution {
    public int kthSmallest(TreeNode root, int k) {
        ArrayList<Integer> list = new ArrayList<>();
        inorder(root, list);
        return list.get(k - 1);
    }

    public void inorder(TreeNode root, ArrayList<Integer> list) {
        if (root == null) return;
        inorder(root.left, list);
    }
}

```

```

        list.add(root.val);
        inorder(root.right, list);
    }
}

```

Validate Binary Search Tree :

```

/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    boolean result;
    public void find(TreeNode root, long min, long max){
        if (root == null) return;

        if (root.val <= min || root.val >= max){
            result = false;
            return;
        }

        find(root.left, min, root.val);
        find(root.right, root.val, max);
    }
    public boolean isValidBST(TreeNode root) {
        result = true;
        find(root, Long.MIN_VALUE, Long.MAX_VALUE);
        return result;
    }
}

```